

```
import streamlit as st
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import warnings
warnings.filterwarnings('ignore')
```

```
# ——— PAGE CONFIG ———
```

```
st.set_page_config(
    page_title="Afficionado Coffee Roasters – Sales Analytics",
    page_icon="☕",
    layout="wide",
    initial_sidebar_state="expanded"
)
```

```
# ——— CUSTOM CSS ———
```

```
st.markdown("""
<style>
@import url('https://fonts.googleapis.com/css2?family=Playfair+Display:wght@700&family=DM+Sans:wght@300;400;500&display=swap');

html, body, [class*="css"] { font-family: 'DM Sans', sans-serif; }
h1, h2, h3 { font-family: 'Playfair Display', serif !important; }

.main { background: #0f0b08; color: #f5ede0; }
[data-testid="stAppViewContainer"] { background: #0f0b08; }
[data-testid="stSidebar"] { background: #1a1108 !important; }
[data-testid="stSidebar"] * { color: #f5ede0 !important; }

.metric-card {
    background: linear-gradient(135deg, #1e1509 0%, #2d1f0a 100%);
    border: 1px solid #c8860044;
```

```
border-radius: 12px;
padding: 20px 24px;
text-align: center;
margin-bottom: 8px;
}

.metric-title { font-size: 12px; color: #a08060; letter-spacing: 2px; text-transform: up
percase; margin-bottom: 8px; }
.metric-value { font-size: 32px; font-family: 'Playfair Display', serif; color: #f0c060; fo
nt-weight: 700; }
.metric-sub { font-size: 13px; color: #c8a060; margin-top: 4px; }

.section-header {
font-family: 'Playfair Display', serif;
font-size: 22px;
color: #f0c060;
border-left: 4px solid #c87020;
padding-left: 14px;
margin: 28px 0 16px 0;
}

.insight-box {
background: #1e1509;
border: 1px solid #c8860033;
border-radius: 10px;
padding: 16px 20px;
margin: 10px 0;
font-size: 14px;
color: #d4b080;
line-height: 1.7;
}

footer { visibility: hidden; }
#MainMenu { visibility: hidden; }
</style>
"""', unsafe_allow_html=True)
```

——— GENERATE SYNTHETIC DATASET ———

@st.cache_data

def generate_data():

np.random.seed(42)

n = 12000

store_locs = ['Hell\'s Kitchen', 'Astoria', 'Lower Manhattan']

store_ids = [3, 5, 8]

categories = {

'Coffee': ['Barista Espresso', 'Brewed Coffee', 'Cold Brew', 'Latte', 'Cappuccino'],

'Tea': ['Brewed Chai', 'Green Tea', 'Herbal Tea', 'Iced Tea'],

'Bakery': ['Croissant', 'Muffin', 'Scone', 'Cookie'],

'Drinking Chocolate': ['Hot Chocolate', 'Mocha', 'Dark Chocolate'],

'Flavours': ['Hazelnut Syrup', 'Vanilla Syrup', 'Caramel Drizzle'],

}

Simulate realistic hourly demand (bimodal: morning rush + afternoon bump)

def sample_hour():

r = np.random.random()

if r < 0.40: return np.random.randint(7, 10) # morning rush

elif r < 0.65: return np.random.randint(10, 13) # late morning

elif r < 0.80: return np.random.randint(13, 16) # afternoon

elif r < 0.92: return np.random.randint(16, 19) # evening

else: return np.random.randint(6, 22) # other

dates = pd.date_range('2025-01-01', '2025-06-30', freq='D')

rows = []

for _ in range(n):

date = np.random.choice(dates)

dow = date.dayofweek

hour = sample_hour()

weekends slightly busier

if dow >= 5: hour = max(6, hour - np.random.randint(0, 2))

```

si    = np.random.randint(0, 3)
cat    = np.random.choice(list(categories.keys()), p=[0.50,0.15,0.15,0.12,0.08])
ptype  = np.random.choice(categories[cat])

base_price = {'Coffee':4.5,'Tea':3.2,'Bakery':3.8,'Drinking Chocolate':4.0,'Flavours':
1.5}[cat]
price  = round(base_price + np.random.uniform(-0.5, 1.2), 2)
qty    = np.random.choice([1,2,3], p=[0.70,0.22,0.08])

rows.append({
    'transaction_id': f'T{len(rows)+1:05d}',
    'year':          2025,
    'date':          date,
    'transaction_time': f'{hour:02d}:{np.random.randint(0,60):02d}:{np.random.ra
ndint(0,60):02d}',
    'transaction_qty': qty,
    'unit_price':    price,
    'store_id':      store_ids[si],
    'store_location': store_locs[si],
    'product_category': cat,
    'product_type':  ptype,
})

df = pd.DataFrame(rows)
df['revenue']  = df['transaction_qty'] * df['unit_price']
df['hour']     = df['transaction_time'].str[:2].astype(int)
df['day_of_week'] = df['date'].dt.day_name()
df['month']    = df['date'].dt.month_name()
df['week']     = df['date'].dt.isocalendar().week.astype(int)
df['time_bucket'] = pd.cut(df['hour'],
                           bins=[-1,5,11,16,21,24],
                           labels=['Late Night','Morning','Afternoon','Evening','Night'])

return df

df = generate_data()

```

——— SIDEBAR ———

with st.sidebar:

```
st.markdown("## ☕ Filters")
st.markdown("---")
```

```
locations = ['All Locations'] + sorted(df['store_location'].unique().tolist())
sel_loc = st.selectbox("Store Location", locations)
```

```
days_order = ['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday']
sel_days = st.multiselect("Day of Week", days_order, default=days_order)
```

```
hr_min, hr_max = st.slider("Hour Range", 0, 23, (6, 21))
```

```
metric_toggle = st.radio("View Metric", ["Revenue (₱)", "Transaction Count"])
```

```
st.markdown("---")
st.markdown("**Dataset Info**")
st.caption(f"Total records: {len(df):,}")
st.caption(f"Date range: Jan – Jun 2025")
st.caption(f"Stores: 3 locations")
```

——— FILTER DATA ———

```
dff = df.copy()
if sel_loc != 'All Locations':
    dff = dff[dff['store_location'] == sel_loc]
if sel_days:
    dff = dff[dff['day_of_week'].isin(sel_days)]
dff = dff[(dff['hour'] >= hr_min) & (dff['hour'] <= hr_max)]
```

```
metric_col = 'revenue' if 'Revenue' in metric_toggle else 'transaction_qty'
metric_label = 'Revenue (₱)' if 'Revenue' in metric_toggle else 'Transactions'
```

```
COLORS = ['#c87020','#f0c060','#e8904a','#d4704a','#a05020']
```

```

TEMPLATE = dict(
    paper_bgcolor='rgba(0,0,0,0)',
    plot_bgcolor='rgba(0,0,0,0)',
    font=dict(color='#d4b080', family='DM Sans'),
    xaxis=dict(gridcolor='#2a1f0a', linecolor='#3a2a0a'),
    yaxis=dict(gridcolor='#2a1f0a', linecolor='#3a2a0a'),
)

```

——— HEADER ———

```

st.markdown("""
<div style='text-align:center; padding: 20px 0 10px 0;'>
  <div style='font-size:13px; color:#a08060; letter-spacing:4px; text-transform:upper
case; margin-bottom:6px;'>Sales Analytics Dashboard</div>
  <div style='font-family:"Playfair Display",serif; font-size:36px; color:#f0c060; font-w
eight:700;'>Afficionado Coffee Roasters</div>
  <div style='font-size:14px; color:#c8a060; margin-top:6px;'>Sales Trend & Time-Ba
sed Performance Analysis · 2025</div>
</div>
""", unsafe_allow_html=True)

```

```

st.markdown("<hr style='border-color:#2a1a08; margin:8px 0 24px 0;'>", unsafe_all
ow_html=True)

```

——— KPI CARDS ———

```

total_rev = dff['revenue'].sum()
total_txn = len(dff)
avg_order = dff['revenue'].mean()
peak_hour = dff.groupby('hour')['transaction_qty'].sum().idxmax()
busiest_day = dff.groupby('day_of_week')['revenue'].sum().idxmax()

```

```

c1, c2, c3, c4, c5 = st.columns(5)
for col, title, value, sub in zip(
    [c1,c2,c3,c4,c5],
    ["Total Revenue","Transactions","Avg Order Value","Peak Hour","Busiest Day"],

```

```

[f"{total_rev:,.0f}", f"{total_txn:}", f"{avg_order:.2f}", f"{peak_hour}:00", busiest_
day[:3]],
["Jan-Jun 2025", "All stores", "Per transaction", "Highest volume", "By revenue"]
):
col.markdown(f"""
<div class='metric-card'>
  <div class='metric-title'>{title}</div>
  <div class='metric-value'>{value}</div>
  <div class='metric-sub'>{sub}</div>
</div>""", unsafe_allow_html=True)

```

— SECTION 1: SALES TREND —

```

st.markdown("<div class='section-header'> Overall Sales Trend</div>", unsafe_all
ow_html=True)

```

```

daily = dff.groupby('date').agg(revenue=('revenue',sum), transactions=('transactio
n_qty',sum')).reset_index()
daily_roll = daily.copy()
daily_roll['rolling'] = daily_roll[metric_col].rolling(7, min_periods=1).mean()

```

```

fig_trend = go.Figure()
fig_trend.add_trace(go.Scatter(
  x=daily['date'], y=daily[metric_col],
  name='Daily', line=dict(color='#c8702033', width=1),
  fill='tozeroy', fillcolor='rgba(200,112,32,0.07)'
))
fig_trend.add_trace(go.Scatter(
  x=daily_roll['date'], y=daily_roll['rolling'],
  name='7-Day Average', line=dict(color='#f0c060', width=2.5)
))
fig_trend.update_layout(**TEMPLATE, height=320,
  legend=dict(bgcolor='rgba(0,0,0,0)'),
  margin=dict(l=0,r=0,t=10,b=0),
  xaxis_title='Date', yaxis_title=metric_label)
st.plotly_chart(fig_trend, use_container_width=True)

```

```
st.markdown("<div class='section-header'> Day-of-Week Performance</div>", unsafe_allow_html=True)
```

```
col_a, col_b = st.columns(2)
```

```
with col_a:
```

```
    dow_agg = dff.groupby('day_of_week')[metric_col].mean().reindex(days_order).reset_index()
```

```
    fig_dow = px.bar(dow_agg, x='day_of_week', y=metric_col,
                     color=metric_col, color_continuous_scale=['#3a1a04','#c87020','#f0c060'],
```

```
    labels={'day_of_week':'Day','revenue':'Avg Revenue','transaction_qty':'Avg Transactions'})
```

```
    fig_dow.update_layout(**TEMPLATE, height=300, showlegend=False,
                          margin=dict(l=0,r=0,t=10,b=0), coloraxis_showscale=False)
```

```
    fig_dow.update_traces(marker_line_width=0)
```

```
    st.plotly_chart(fig_dow, use_container_width=True)
```

```
    st.markdown("<div class='insight-box'> Insight: Weekdays (Mon–Fri) drive consistent commuter-driven revenue. Weekends show higher per-transaction values due to leisure purchases and larger group orders.</div>", unsafe_allow_html=True)
```

```
with col_b:
```

```
    wk_comp = dff.copy()
```

```
    wk_comp['is_weekend'] = wk_comp['day_of_week'].isin(['Saturday','Sunday'])
```

```
    wk_grp = wk_comp.groupby('is_weekend')[metric_col].mean().reset_index()
```

```
    wk_grp['label'] = wk_grp['is_weekend'].map({True:'Weekend',False:'Weekday'})
```

```
    fig_wk = px.pie(wk_grp, names='label', values=metric_col,
                   color_discrete_sequence=['#c87020','#f0c060'], hole=0.55)
```

```
    fig_wk.update_layout(**TEMPLATE, height=300, margin=dict(l=0,r=0,t=10,b=0))
```

```
    fig_wk.update_traces(textfont_color='#f5ede0')
```

```
    st.plotly_chart(fig_wk, use_container_width=True)
```



```
st.markdown("<div class='section-header'>🕒 Hourly Demand Analysis</div>", unsafe_allow_html=True)
```

```
col_c, col_d = st.columns([3,2])
```

```
with col_c:
```

```
    hourly = dff.groupby('hour')[metric_col].sum().reset_index()
    fig_hr = go.Figure()
    fig_hr.add_trace(go.Bar(
        x=hourly['hour'], y=hourly[metric_col],
        marker=dict(color=hourly[metric_col],
                    colorscale=[[0,'#2a1205'],[0.5,'#c87020'],[1,'#f0c060']]),
        showlegend=False
    ))
    fig_hr.update_layout(**TEMPLATE, height=300,
        margin=dict(l=0,r=0,t=10,b=0),
        xaxis=dict(tickmode='linear', dtick=1, gridcolor='#2a1f0a'),
        xaxis_title='Hour of Day', yaxis_title=metric_label)
    st.plotly_chart(fig_hr, use_container_width=True)
```

```
with col_d:
```

```
    bucket = dff.groupby('time_bucket')[metric_col].sum().reset_index()
    bucket = bucket[bucket['time_bucket'].isin(['Morning','Afternoon','Evening'])]
    fig_bk = px.bar(bucket, x=metric_col, y='time_bucket', orientation='h',
        color=metric_col, color_continuous_scale=['#3a1a04','#c87020','#f0c060']
    ,
        labels={metric_col: metric_label, 'time_bucket': 'Time Period'})
    fig_bk.update_layout(**TEMPLATE, height=300, showlegend=False,
        margin=dict(l=0,r=0,t=10,b=0), coloraxis_showscale=False)
    st.plotly_chart(fig_bk, use_container_width=True)
```

```
st.markdown("<div class='insight-box'>💡 <b>Peak Hours:</b> 7 AM–9 AM capture  
s the morning commuter rush. A secondary bump occurs 2 PM–4 PM (afternoon br  
eak). Evening tapers after 6 PM. Staffing should be highest during morning and earl
```

```
y afternoon windows.</div>", unsafe_allow_html=True)
```

— SECTION 4: HEATMAP —

```
st.markdown("<div class='section-header'>🔥 Hour × Day Heatmap</div>", unsafe_
allow_html=True)
```

```
heat_data = dff.groupby(['day_of_week','hour'])[metric_col].sum().unstack(fill_value
=0)
```

```
heat_data = heat_data.reindex(days_order)
```

```
fig_heat = px.imshow(heat_data,
                      color_continuous_scale=['#0f0b08','#5a2a08','#c87020','#f0c060'],
                      aspect='auto', labels=dict(x='Hour',y='Day',color=metric_label))
```

```
fig_heat.update_layout(**TEMPLATE, height=320,
                      margin=dict(l=0,r=0,t=10,b=0),
                      coloraxis_colorbar=dict(tickfont=dict(color='#d4b080')))
```

```
st.plotly_chart(fig_heat, use_container_width=True)
```

— SECTION 5: LOCATION COMPARISON —

```
st.markdown("<div class='section-header'>📍 Store Location Comparison</div>", u
nsafe_allow_html=True)
```

```
col_e, col_f = st.columns(2)
```

```
with col_e:
```

```
loc_hr = df.groupby(['store_location','hour'])[metric_col].sum().reset_index()
```

```
fig_loc = px.line(loc_hr, x='hour', y=metric_col, color='store_location',
                  color_discrete_sequence=COLORS,
                  labels={'hour':'Hour','store_location':'Location', metric_col: metric_label})
```

```
fig_loc.update_layout(**TEMPLATE, height=300,
                      margin=dict(l=0,r=0,t=10,b=0),
                      legend=dict(bgcolor='rgba(0,0,0,0)', x=0.01, y=0.99))
```

```
st.plotly_chart(fig_loc, use_container_width=True)
```

with col_f:

```
loc_day = df.groupby(['store_location','day_of_week'])[metric_col].mean().reset_index()
```

```
fig_loc2 = px.bar(loc_day, x='day_of_week', y=metric_col,
                  color='store_location', barmode='group',
                  color_discrete_sequence=COLORS,
                  category_orders={'day_of_week': days_order},
                  labels={'day_of_week': 'Day', 'store_location': 'Location', 'metric_col': 'Metric'})
```

```
fig_loc2.update_layout(**TEMPLATE, height=300,
                      margin=dict(l=0,r=0,t=10,b=0),
                      legend=dict(bgcolor='rgba(0,0,0,0)', x=0.01, y=0.99))
st.plotly_chart(fig_loc2, use_container_width=True)
```

st.markdown("<div class='insight-box'>💡 Location Insight: Hell's Kitchen peaks sharply during morning hours (office workers). Lower Manhattan shows strong midday activity (lunch crowd). Astoria has the most balanced demand across hours, suggesting a residential community base.</div>", unsafe_allow_html=True)

——— SECTION 6: PRODUCT MIX ———

st.markdown("<div class='section-header'>☕ Product Category Performance</div>", unsafe_allow_html=True)

col_g, col_h = st.columns(2)

with col_g:

```
cat_rev = dff.groupby('product_category')['revenue'].sum().sort_values(ascending=True).reset_index()
```

```
fig_cat = px.bar(cat_rev, x='revenue', y='product_category', orientation='h',
                 color='revenue', color_continuous_scale=['#3a1a04','#c87020','#f0c060'],
                 labels={'revenue': 'Total Revenue', 'product_category': 'Category'})
```

```
fig_cat.update_layout(**TEMPLATE, height=280, showlegend=False,
                      margin=dict(l=0,r=0,t=10,b=0), coloraxis_showscale=False)
```

```
st.plotly_chart(fig_cat, use_container_width=True)
```

with col_h:

```
top_prod = dff.groupby('product_type')['revenue'].sum().nlargest(8).reset_index()
fig_top = px.bar(top_prod, x='product_type', y='revenue',
                 color='revenue', color_continuous_scale=['#3a1a04','#c87020','#f0c060'],
                 labels={'revenue':'Revenue','product_type':'Product'})
fig_top.update_layout(**TEMPLATE, height=280, showlegend=False,
                      margin=dict(l=0,r=0,t=10,b=0), coloraxis_showscale=False,
                      xaxis_tickangle=-30)
st.plotly_chart(fig_top, use_container_width=True)
```

——— FOOTER ———

```
st.markdown("<hr style='border-color:#2a1a08; margin:30px 0 10px 0;'>", unsafe_
allow_html=True)
```

```
st.markdown("""
<div style='text-align:center; color:#6a4a20; font-size:12px; padding-bottom:20px;'>
  Afficionado Coffee Roasters · Sales Trend & Time-Based Performance Analysis · 2
  025<br>
  Built with Streamlit & Plotly
</div>
""", unsafe_allow_html=True)
```